FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	Faith S. Nemeno
Section:	AN43
Professor:	Ms. Crisola Tan
Date:	November 2, 2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

```
Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella
```

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. **Which system (RBR or CBR) was easier to implement? Why?**

   RBR was easier to implement because it requires predefined conditions, such as product category, the information, and a simple yes or no preference from the customer. Because of this the system just followed these rules to recommend a product, but the final decision still comes from the customer. Compared to CBR, it required less logic since there was no need to calculate similarities of cases.

2. **Which is more flexible in solving unstructured problems?**

   CBR is more flexible because it uses the previous cases as references to solve new problems. Even though it has collections of existing solutions, the customer preferences will still be different. Instead of relying on predefined rules, the system looks for similar past cases and uses that as a starting point to find a solution for the customer's needs.

3. **Which is better for situations with continuously changing data?**

   For a cafe menu, I think RBR is more suitable for managing a large and frequently updated menu because new products are added every now and then. But for customer preferences, CBR will be more suitable since it has a touch of personalization since it uses past preferences or orders. Both methods have their advantages depending on where it is used.

4. **Examples of real-world applications for both.**

   A real-world application would be:

   RBR: A room assignment program for hotels or hospitals. The system can follow predefined conditions like room availability, capacity, and other requirements catered to the customers.

   CBR: For law firms, the system can refer to previous cases with similar circumstances and use that as reference when handling new cases.



Your Code - Laboratory Tasks

```
# task 1A

categories = {
  "pasta",
  "coffee",
  "pastry"
}


# facts
# knowledge base
menu = {

  #pasta
  "Carbonara": {
    "category": "pasta",
    "price": 180,
    "serving": 2,
    "dine-in": True,
    "availability": True
  },

  "Pesto": {
    "category": "pasta",
    "price": 190,
    "serving": 2,
    "dine-in": True,
    "availability": True
  },

  #coffee

  "Americano": {
    "category": "coffee",
    "price": 90,
    "serving": 1,
    "dine-in": True,
    "availability": True
  },

 "Cafe Latter": {
    "category": "coffee",
    "price": 100,
    "serving": 1,
    "dine-in": True,
    "availability": False
  },

  #pastry
```

```
  "Croissant": {
    "category": "pastry",
    "price": 85,
    "serving": 1,
    "dine-in": True,
    "availability": False
  },

 "Brownies": {
    "category": "pastry",
    "price": 65,
    "serving": 1,
    "dine-in": True,
    "availability": True
  },
}

# display menu

print("       Cafe Menu")
print("_________________________")

print("\nProduct Categories:", categories)

for product, details in menu.items():
  print(f"\nProduct: {product}")

  for attribute, value in details.items():
    print(f"{attribute}: {value}")

print("\nTotal Products:", len(menu))

# RBR

# rule 1 : coffee reccomendation
def rule_coffee(product, preferences):
  if (preferences["wants_coffee"] and product["category"] == "coffee"):
    return True

  return False

# rule 2 : pastry recommendation
def rule_pastry(product, preferences):
  if (preferences["wants_pastry"] and product["category"] == "pastry"):
    return True

  return False

# rule 3 : pasta recommendation
```

```
def rule_pasta(product, preferences):
  if (preferences["wants_pasta"] and product["category"] == "pasta"):
    return True

  return False

# inference
def inference_engine(menu, preferences):

  recommendations = []

  # check menu
  for product_name, details in menu.items():

    # ignore unavail prods
    if not details["availability"]:
      continue

    # check all rules
    if rule_coffee(details, preferences):
      recommendations.append(product_name)

    elif rule_pastry(details, preferences):
      recommendations.append(product_name)

    elif rule_pasta(details, preferences):
      recommendations.append(product_name)

  return recommendations

# input

print("     Customer Order")
print("_____________________")

preferences = {
    "wants_coffee": input("Do you want coffee? (y/n): ").lower() == "y",
    "wants_pastry": input("Do you want pastry? (y/n): ").lower() == "y",
    "wants_pasta": input("Do you want pasta? (y/n): ").lower() == "y"
}

# display reccoss

results = inference_engine(menu, preferences)

print("Rule-Based Recommendations")
print("__________________________")

if results:
  for item in results:
    print(f"- {item} (P{menu[item]['price']})")
```

```
else:
  print("no recommendations found.")

print("\ntotal recommendations:", len(results))

# CBR

# previous orders
case_base = [

    {
        "problem": {
            "wants_coffee": True,
            "wants_pastry": True,
            "wants_pasta": False
        },
        "solution": ["Cafe Latte", "Brownies"]
    },

    {
        "problem": {
            "wants_coffee": True,
            "wants_pastry": False,
            "wants_pasta": False
        },
        "solution": ["Americano"]
    },

    {
        "problem": {
            "wants_coffee": False,
            "wants_pastry": False,
            "wants_pasta": True
        },
        "solution": ["Croissant"]
    },
]


# retrieve most similar previous case

def retrieve_case(case_base, new_problem):

    most_similar_case = None
    highest_score = -1

    for case in case_base:

        matching_features = 0
```

```
        for feature in new_problem:

            if case["problem"][feature] == new_problem[feature]:
                matching_features += 1

        similarity = matching_features / len(new_problem)

        if similarity > highest_score:
            highest_score = similarity
            most_similar_case = case

    return most_similar_case, highest_score


# reuse solution from the retrieved case

def reuse_solution(retrieved_case):

    if retrieved_case is None:
        return []

    return retrieved_case["solution"].copy()

# revise solutions by modifying reccoss

def revise_solution(solution, menu):

    print("\nCurrent Recommendation:", ", ".join(solution))

    choice = input("Would you like to revise this recommendation? (y/n):
").lower()

    if choice == "y":

        print("\nAvailable Products:")

        for product in menu:
            if menu[product]["availability"]:
                print("-", product)

        new_items = input(
            "\nEnter your preferred products separated by commas: "
        )

        revised_solution = [
            item.strip() for item in new_items.split(",")
            if item.strip() in menu and menu[item.strip()]["availability"]
        ]

        if revised_solution:
            return revised_solution
```

```
        print("No valid products entered. Keeping original recommendation.")

    return solution

# retain or store the new case for future recommendations

def retain_case(case_base, problem, solution):

    new_case = {
        "problem": problem.copy(),
        "solution": solution.copy()
    }

    case_base.append(new_case)

    print("\nNew case successfully added to the case base!")


# display cbr

print("Case-Based Recommendations")
print("_____________________________")

# retreive
retrieved_case, score = retrieve_case(case_base, preferences)

print("\n[RETRIEVE]")
print("Most similar case found!")
print("Similarity Score:", round(score * 100, 2), "%")

# reuse
recommendations = reuse_solution(retrieved_case)

print("\n[REUSE]")
print("Previous Solution:", recommendations)

# revise
final_recommendations = revise_solution(recommendations, menu)

print("\n[REVISE]")
print("Final Recommendation:", final_recommendations)

# retain
retain_case(case_base, preferences, final_recommendations)

# updated case base
print("    Updated Case-Based")
print("_____________________________")
```

```
print("Total Cases:", len(case_base))

for index, case in enumerate(case_base, start=1):
    print(f"\nCase {index}")
    print("Problem:", case["problem"])
    print("Solution:", case["solution"])
```

RBR:

```
          Cafe Menu
 ...  ____________________

    Product Categories: {'pastry', 'coffee', 'pasta'}

    Product: Carbonara
    category: pasta
    price: 180
    serving: 2
    dine-in: True
    availability: True

    Product: Pesto
    category: pasta
    price: 190
    serving: 2
    dine-in: True
    availability: True

    Product: Americano
    category: coffee
    price: 90
    serving: 1
    dine-in: True
    availability: True

    Product: Cafe Latter
    category: coffee
    price: 100
    serving: 1
    dine-in: True
    availability: False

    Product: Croissant
    category: pastry
    price: 85
    serving: 1
    dine-in: True
    availability: False

    Product: Brownies
    category: pastry
    price: 65
    serving: 1
    dine-in: True
    availability: True

    Total Products: 6
         Customer Order
    ____________________
    Do you want coffee? (y/n): n
    Do you want pastry? (y/n): y
    Do you want pasta? (y/n): n
    Rule-Based Recommendations
    ____________________
    - Brownies (P65)

    total recommendations: 1
```

CBR:

```
Case-Based Recommendations
_____________________

[RETRIEVE]
Most similar case found!
Similarity Score: 66.67 %

[REUSE]
Previous Solution: ['Cafe Latte', 'Brownies']

Current Recommendation: Cafe Latte, Brownies
Would you like to revise this recommendation? (y/n): y

Available Products:
- Carbonara
- Pesto
- Americano
- Brownies

Enter your preferred products separated by commas: Pesto

[REVISE]
Final Recommendation: ['Pesto']

New case successfully added to the case base!
    Updated Case-Based
_____________________
Total Cases: 4

Case 1
Problem: {'wants_coffee': True, 'wants_pastry': True, 'wants_pasta': False}
Solution: ['Cafe Latte', 'Brownies']

Case 2
Problem: {'wants_coffee': True, 'wants_pastry': False, 'wants_pasta': False}
Solution: ['Americano']

Case 3
Problem: {'wants_coffee': False, 'wants_pastry': False, 'wants_pasta': True}
Solution: ['Croissant']

Case 4
Problem: {'wants_coffee': False, 'wants_pastry': True, 'wants_pasta': False}
Solution: ['Pesto']
```